FOR HUTARKA TUTORS

How to write a Hutarka lesson

Each lesson is one JSON file. The app reads it as it is, so what you write is exactly what learners see and hear. This page explains every field, shows a finished lesson, and ends with a checklist.

How a lesson works

Hutarka teaches English through short everyday situations. A learner plays a boar, and in every lesson the boar has a conversation with another character: ordering coffee from a fox, asking an owl for directions. A lesson takes about five minutes and runs in five stages, always in this order. Each stage comes from one part of your file.

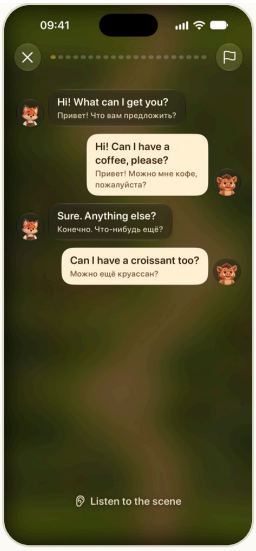
1. **Scene**
   The learner watches the whole conversation, line by line, with translations. From `scene`.

2. **New words**
   A card for each word, then a quick quiz: match, listen and pick, spell. From `words`.

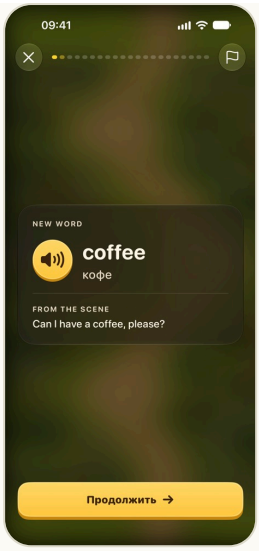
3. **Phrases**
   Reusable chunks, learned whole: fill the gap, build from tiles, spot the mistake. From `phrases`.

4. **Swap**
   One sentence pattern with a changing word: "Can I have a ___, please?" From `swap`.

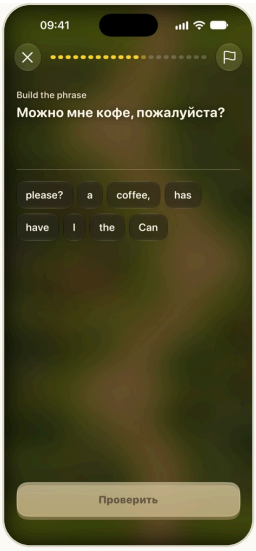
5. **Your turn**
   The scene again, but now the learner says the boar's lines, by building them or picking the right one. From each boar line's `task`.



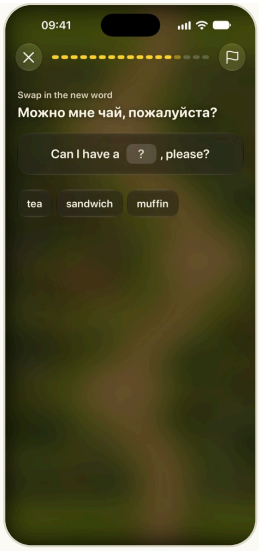
1 Scene. The conversation plays line by line.



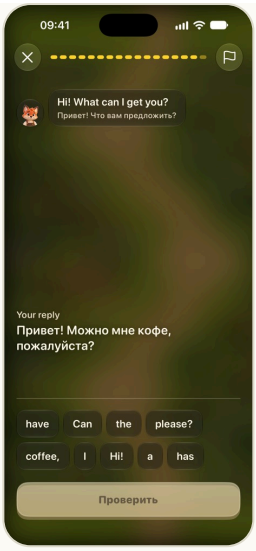
2 New words. One card per word.



3 Phrases. Built from tiles, among other exercises.



4 Swap. New words in the same pattern.



5 Your turn. The learner says the boar's lines.

The app decides which exercise to use each time, so every session plays a little differently. You don't write exercises; you write the material they are made from. The screenshots below all come from the café lesson in the full example, so you can match each field to what it becomes.

The files

- **One file per lesson**, named after its id: `lessons/cafe.json`, `lessons/restaurant.json`. This is where nearly all your work goes.
- **One** `trail.json` for the order of lessons on the learner's map. It only lists lesson ids and names, and is usually prepared for you.

Save files as UTF-8 so Cyrillic, accents and other scripts come through. Use straight quotes in JSON (`"`), and curly ones only inside the text itself if you want them.

Writing text

Every piece of text a learner reads is an object with one entry per language:

```
{ "en": "Here you go.", "ru": "Вот, пожалуйста.", "es": "Aquí tiene." }
```

- `en` is required: it's the English being taught.
- The other entries are what it means in the learner's own language. Translate the meaning, the way a native speaker would say it in the same situation.
- A missing language shows the English instead, so you can deliver the English and a few languages first, and others later.

Language codes

`es` Spanish `fr` French `de` German `it` Italian `pt` Portuguese `ru` Russian `uk` Ukrainian `be` Belarusian `pl` Polish `tr` Turkish `zh` Chinese (Simplified) `zh-Hant` Chinese (Traditional) `ja` Japanese `ar` Arabic `hi` Hindi

Wrong answers and extra tiles are the exception: they are plain lists of English strings, like `["has", "the"]`, because learners only ever see them in English.

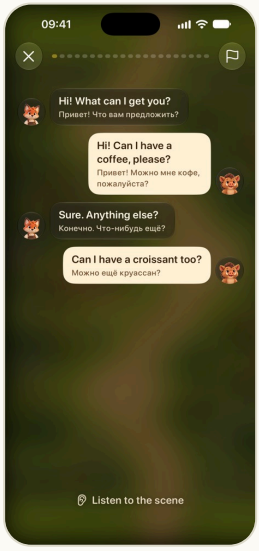
Lesson fields

About the lesson

FIELD	WHAT IT IS	RULES
`id`	The lesson's name for the app.	Lowercase with dashes, the same as the file name: `the-bill`.
`title`	Shown on the map, e.g. "Café".	Text object. One or two words.
`subtitle`	What the learner will be able to do.	Text object. Starts with a verb: "Order a coffee and a snack".
`with`	Who the boar talks to.	One of `fox`, `owl`, `dog`, `rabbit`.



title and **subtitle** on the map's card: "Café", "Order a coffee and a snack".

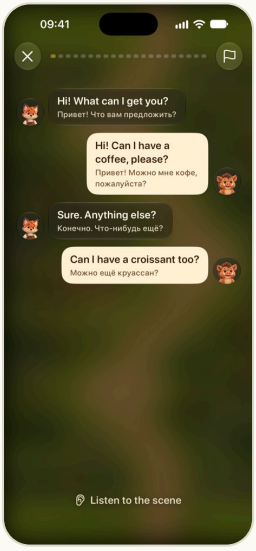


with: "fox" puts the fox beside the other character's lines.

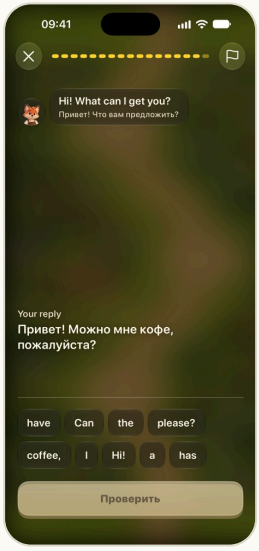
scene

The conversation, in order. 4 to 8 lines, starting with the other character, taking turns.

FIELD	WHAT IT IS	RULES
`speaker`	Who says the line.	`boar` (the learner) or `other`.
`text`	The line.	Text object. Short enough to say in one breath.
`task`	Boar lines only: how the learner says it in "Your turn".	Required on every boar line, absent on the other character's. Exactly one of the two below.
`task.build`	The learner builds the line from its words, plus these extra tiles.	2 English words that are *not* in the line but look plausible: `["has", "the"]`.
`task.choose`	The learner picks the line from a list that also has these wrong replies.	2 sentences that are close but wrong for one clear reason.



Each line's **text**: English on top, the learner's language underneath.



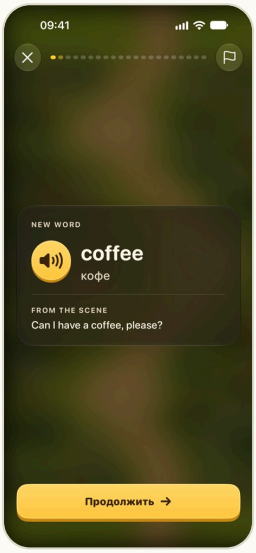
task.build in "Your turn": the line's own tiles plus your extras, **has** and **the**.

words

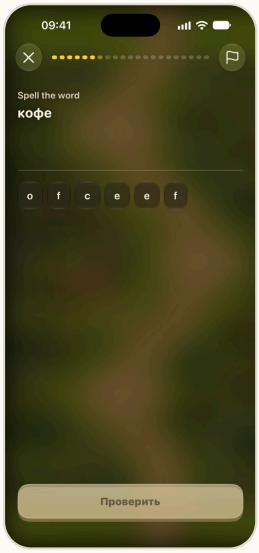
4 to 6 new words or short fixed expressions, each taken from the scene.

FIELD	WHAT IT IS	RULES
`text`	The word, in lowercase unless it's a name.	Text object: `{ "en": "croissant", "ru": "круассан" }`.
`example`	The scene line it comes from, shown on the word's card.	A plain English string, copied **exactly** from a scene line, punctuation included.

Single words of 8 letters or fewer may also be spelled letter by letter in a session, so check their spelling twice.



The word's **text**, and its **example** under "From the scene".

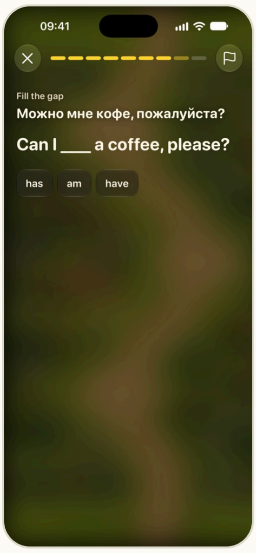


"Spell the word": short words get split into letters.

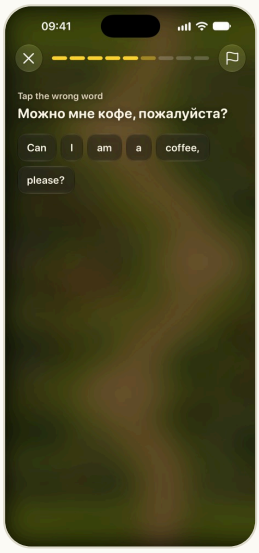
phrases

2 to 4 chunks the learner should be able to reuse anywhere, ideally lines or parts of lines from the scene.

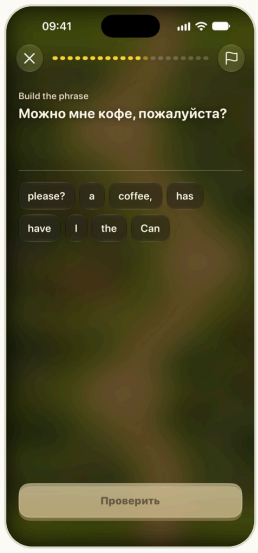
FIELD	WHAT IT IS	RULES
`text`	The phrase.	Text object.
`distractors`	Extra tiles when building the phrase.	2 English words not in the phrase.
`gap`	The word that's blanked out in "fill the gap", and replaced in "spot the mistake".	One whole word of the phrase that appears only once in it. Avoid tiny words like `a` or `I`: they hide inside other words.
`gapWrong`	Wrong options offered for the gap.	2 English words. One of them also becomes the mistake in "spot the mistake", so it must be clearly wrong in that place.



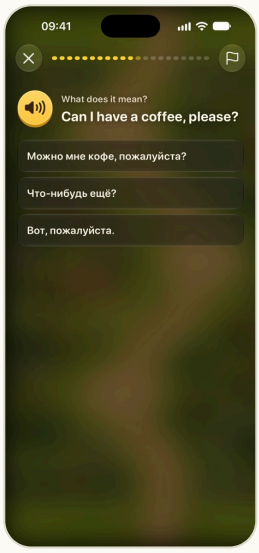
gap "have" is blanked out; **gapWrong** "has" and "am" are the other options.



"Tap the wrong word": the **gap** word swapped for one of **gapWrong**.



"Build the phrase": its words plus **distractors**. Punctuation stays on the word: **please?**

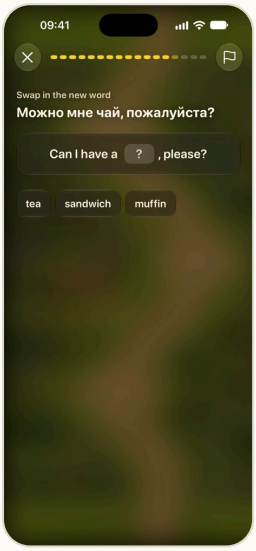


"What does it mean?": the phrase's translation among the lesson's other phrases.

swap

One sentence pattern with a slot, and 3 words that fit it.

FIELD	WHAT IT IS	RULES
`frame`	The pattern, with `{slot}` where the word goes.	Plain English string with `{slot}` exactly once: `"Can I have a {slot}, please?"`
`items[].word`	A word that fits the slot.	Text object. Not one of the scene's own words: this is where the learner stretches the pattern.
`items[].sentence`	The full sentence with that word in.	Text object. Its `en` must equal the frame with the word filled in, letter for letter.



The **frame** with its slot, the **sentence**'s translation on top, and the three **word**s to pick from.

Watch articles in the frame. "Can I have a {slot}" works for *tea* and *muffin* but not for *apple* or *water*. Choose words that fit the frame as written.

What you don't write

The app works these out itself: splitting lines into tiles, which exercises to show, the sentence with a mistake in it, audio (read aloud by the device), and the word and phrase counts on the map.

The trail file

The map is made of chapters. Each chapter has 5 units, each unit is one illustrated map with 3 lessons on it, listed from the bottom stump up.

trail.json

```
{
  "chapters": [
    {
      "id": "first-steps",
      "title": { "en": "First steps", "ru": "Первые шаги" },
      "units": [
        {
          "title": { "en": "Eating out", "ru": "В кафе" },
          "map": "meadow",
          "lessons": ["cafe", "restaurant", "the-bill"]
        }
      ]
    }
  ]
}
```



Each stump is a lesson; the boar stands on the next one to play.

- `map` is the illustration: `forest`, `meadow` or `farm`.
- The last stump of every chapter is a **checkpoint**: a review the app builds from the chapter's lessons. Write the word `checkpoint` in its place; it has no lesson file. So a chapter is 14 lessons plus the checkpoint.

Full example

The café lesson, as it is in the app today. It has Russian and Spanish translations; other languages would follow the same pattern.

lessons/cafe.json

```
{
  "id": "cafe",
  "title": { "en": "Café", "ru": "Кафе", "es": "Cafetería" },
  "subtitle": { "en": "Order a coffee and a snack", "ru": "Закажите кофе и перекус", "es":
"Pide un café y algo de comer" },
  "with": "fox",
  "scene": [
    {
      "speaker": "other",
      "text": { "en": "Hi! What can I get you?", "ru": "Привет! Что вам предложить?", "es":
"¡Hola! ¿Qué le pongo?" }
    },
    {
      "speaker": "boar",
      "text": { "en": "Hi! Can I have a coffee, please?", "ru": "Привет! Можно мне кофе,
пожалуйста?", "es": "¡Hola! ¿Me pone un café, por favor?" },
      "task": { "build": ["has", "the"] }
```

```
    },
    {
      "speaker": "other",
      "text": { "en": "Sure. Anything else?", "ru": "Конечно. Что-нибудь ещё?", "es": "Claro.
¿Algo más?" }
    },
    {
      "speaker": "boar",
      "text": { "en": "Can I have a croissant too?", "ru": "Можно ещё круассан?", "es": "¿Me
pone también un cruasán?" },
      "task": { "choose": ["I have a croissant too.", "Can you have a croissant?"] }
    },
    {
      "speaker": "other",
      "text": { "en": "Of course. That's five euros.", "ru": "Конечно. С вас пять евро.",
"es": "Por supuesto. Son cinco euros." }
    },
    {
      "speaker": "boar",
      "text": { "en": "Here you go. Thank you!", "ru": "Вот, пожалуйста. Спасибо!", "es":
"Aquí tiene. ¡Gracias!" },
      "task": { "build": ["come", "Thanks"] }
    }
  ],
  "words": [
    { "text": { "en": "coffee", "ru": "кофе", "es": "café" }, "example": "Hi! Can I have a
coffee, please?" },
    { "text": { "en": "croissant", "ru": "круассан", "es": "cruasán" }, "example": "Can I
have a croissant too?" },
    { "text": { "en": "please", "ru": "пожалуйста", "es": "por favor" }, "example": "Hi! Can
I have a coffee, please?" },
    { "text": { "en": "anything else", "ru": "что-нибудь ещё", "es": "algo más" }, "example":
"Sure. Anything else?" },
    { "text": { "en": "here you go", "ru": "вот, пожалуйста", "es": "aquí tiene" },
"example": "Here you go. Thank you!" }
  ],
  "phrases": [
    {
      "text": { "en": "Can I have a coffee, please?", "ru": "Можно мне кофе, пожалуйста?",
"es": "¿Me pone un café, por favor?" },
      "distractors": ["has", "the"],
      "gap": "have",
      "gapWrong": ["has", "am"]
    },
    {
      "text": { "en": "Anything else?", "ru": "Что-нибудь ещё?", "es": "¿Algo más?" },
      "distractors": ["Something", "other"],
      "gap": "else",
      "gapWrong": ["more", "other"]
    },
    {
      "text": { "en": "Here you go.", "ru": "Вот, пожалуйста.", "es": "Aquí tiene." },
      "distractors": ["are", "come"],
      "gap": "go",
      "gapWrong": ["come", "are"]
    }
  ],
```

```
  "swap": {
    "frame": "Can I have a {slot}, please?",
    "items": [
      {
        "word": { "en": "tea", "ru": "чай", "es": "té" },
        "sentence": { "en": "Can I have a tea, please?", "ru": "Можно мне чай, пожалуйста?",
"es": "¿Me pone un té, por favor?" }
      },
      {
        "word": { "en": "muffin", "ru": "маффин", "es": "magdalena" },
        "sentence": { "en": "Can I have a muffin, please?", "ru": "Можно мне маффин,
пожалуйста?", "es": "¿Me pone una magdalena, por favor?" }
      },
      {
        "word": { "en": "sandwich", "ru": "сэндвич", "es": "sándwich" },
        "sentence": { "en": "Can I have a sandwich, please?", "ru": "Можно мне сэндвич,
пожалуйста?", "es": "¿Me pone un sándwich, por favor?" }
      }
    ]
  }
}
```

Blank template

Copy this to start a new lesson. Replace every value, add translations, and add or remove scene lines, words and phrases within the limits above.

lessons/your-lesson-id.json

```
{
  "id": "your-lesson-id",
  "title": { "en": "" },
  "subtitle": { "en": "" },
  "with": "fox",
  "scene": [
    { "speaker": "other", "text": { "en": "" } },
    { "speaker": "boar", "text": { "en": "" }, "task": { "build": ["", ""] } },
    { "speaker": "other", "text": { "en": "" } },
    { "speaker": "boar", "text": { "en": "" }, "task": { "choose": ["", ""] } }
  ],
  "words": [
    { "text": { "en": "" }, "example": "" },
    { "text": { "en": "" }, "example": "" },
    { "text": { "en": "" }, "example": "" },
    { "text": { "en": "" }, "example": "" }
  ],
  "phrases": [
    { "text": { "en": "" }, "distractors": ["", ""], "gap": "", "gapWrong": ["", ""] },
    { "text": { "en": "" }, "distractors": ["", ""], "gap": "", "gapWrong": ["", ""] }
  ],
  "swap": {
    "frame": "... {slot} ...",
    "items": [
```

```
      { "word": { "en": "" }, "sentence": { "en": "" } },
      { "word": { "en": "" }, "sentence": { "en": "" } },
      { "word": { "en": "" }, "sentence": { "en": "" } }
    ]
  }
}
```

Writing well

Keep the boar's lines sayable

Every boar line becomes something the learner builds or chooses. A beginner should be able to say it after the lesson.

GOOD

Can I have a coffee, please?

TOO MUCH

Would it be possible for me to get a large oat latte to go?

Make wrong answers wrong for one reason

The learner should be able to tell why an option is wrong. Nonsense options teach nothing; near-misses do.

GOOD

`gap: "have"`, `gapWrong: ["has", "am"]`

TOO EASY

`gapWrong: ["banana", "blue"]`

Think in tiles

Lines are split into tiles at the spaces, so punctuation stays on its word: "Here you go." becomes `Here` `you` `go.` An extra tile must not be the same as a real one, or the learner can't tell them apart.

Translate the meaning

GOOD

Here you go. → Вот, пожалуйста.

WORD FOR WORD

Here you go. → Здесь ты идёшь.

Teach from the scene

Words and phrases should all appear in the conversation first, so the learner meets them in context before practising them. The swap words are the one place to bring in new vocabulary.

Before you send

Tick these off for each lesson. They're the mistakes that stop a lesson from loading or make an exercise wrong.

- [ ] The file is valid JSON. Paste it into any JSON validator, such as the one in your code editor, to check.
- [ ] The `id` matches the file name, and is lowercase with dashes.
- [ ] The scene starts with the other character, and every boar line has a `task`.
- [ ] Every word's `example` is copied exactly from a scene line.
- [ ] Every phrase's `gap` is a whole word that appears once in that phrase.
- [ ] No extra tile (`build`, `distractors`) is already in the line or phrase it goes with.
- [ ] The swap `frame` has `{slot}` once, and each sentence is the frame with its word filled in.
- [ ] Every text object has `en`, and the translations you added sound natural to a native speaker.

Then send the lesson files back to the Hutarka team.